VL 02 · Foundation Models verstehen

Was ihr über ein Modell wissen müsst, um es als Baustein einzusetzen

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27

Letzte Woche in der Übung

Ihr habt zwei Modelle gefragt: *„Ist die HTWG Konstanz eine private oder eine staatliche Hochschule?“*

Antwort A

„Die HTWG Konstanz ist eine private, staatlich anerkannte Hochschule …“

Antwort B

„Die HTWG Konstanz ist eine staatliche Hochschule für angewandte Wissenschaften …“

Dasselbe Modell, dieselbe Frage, zwei Durchläufe: **nicht immer dieselbe Antwort.**

Frage an euch: Woher kommt die falsche Antwort? Und warum kommt sie nur manchmal? Am Ende der Vorlesung könnt ihr beides erklären.

Vier Entscheidungen prägen jedes Modell

Ihr müsst kein Modell bauen können. Aber ihr müsst wissen, was beim Bauen entschieden wurde, denn daraus folgen **Modellwahl, Kosten und die Einstellungen eures API-Aufrufs**.

📚 1 · Trainingsdaten

Woraus hat das Modell gelernt? Welche Sprachen, welche Fachgebiete fehlen?

🧱 2 · Modell und Größe

Wie ist es aufgebaut, wie viele Parameter hat es, wie viel Speicher braucht es?

🎓 3 · Post-Training

Wie wurde aus einem Textvervollständiger ein hilfreicher Assistent?

🎲 4 · Sampling

Wie wird aus Wahrscheinlichkeiten eine Antwort? **Euer direkter Hebel.**

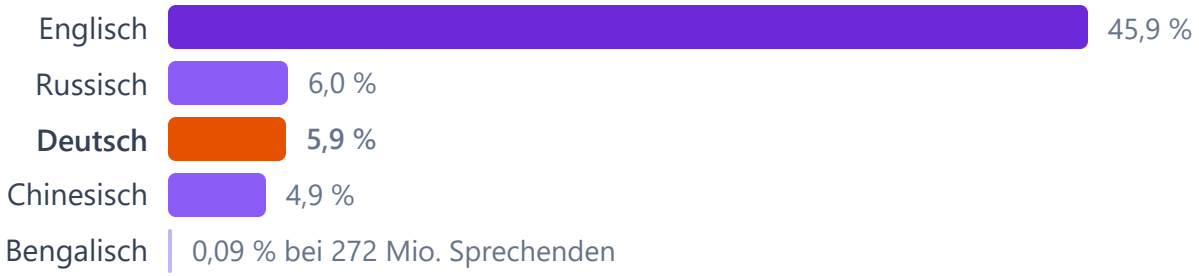
Reihenfolge und Begriffe folgen Huyen, *AI Engineering*, Kapitel 2. Ab 10:45 probiert ihr selbst aus.

1 · Trainingsdaten

Ein Modell ist nur so gut wie die Daten, aus denen es gelernt hat

Was im Web steht, lernt das Modell

Die meisten großen Modelle lernen aus **Common Crawl**, einem Abzug des öffentlichen Webs. Darin stehen Lexika und Fachseiten, aber auch Clickbait und Falschmeldungen. Verteilung der Sprachen:



Deutsch ist auf Platz 3, also **gut vertreten**. Viele Sprachen mit Hunderten Millionen Sprechenden sind dagegen *low-resource*.

Was daraus für euren Assistenten folgt

🎯 Qualität

Modelle sind in Englisch messbar besser. GPT-4 löste Matheaufgaben auf Englisch **dreimal so oft** wie auf Armenisch.

💶 Kosten

Abgerechnet wird pro **Token**. Birmanisch braucht für denselben Inhalt etwa **10×** so viele Tokens wie Englisch. Deutsch auch mehr. Wie viel, messt ihr heute.

🏛️ Domänen

Was nicht öffentlich im Web steht, fehlt: Proteindaten, Röntgenbilder, und die **Prüfungsordnung der HTWG**.

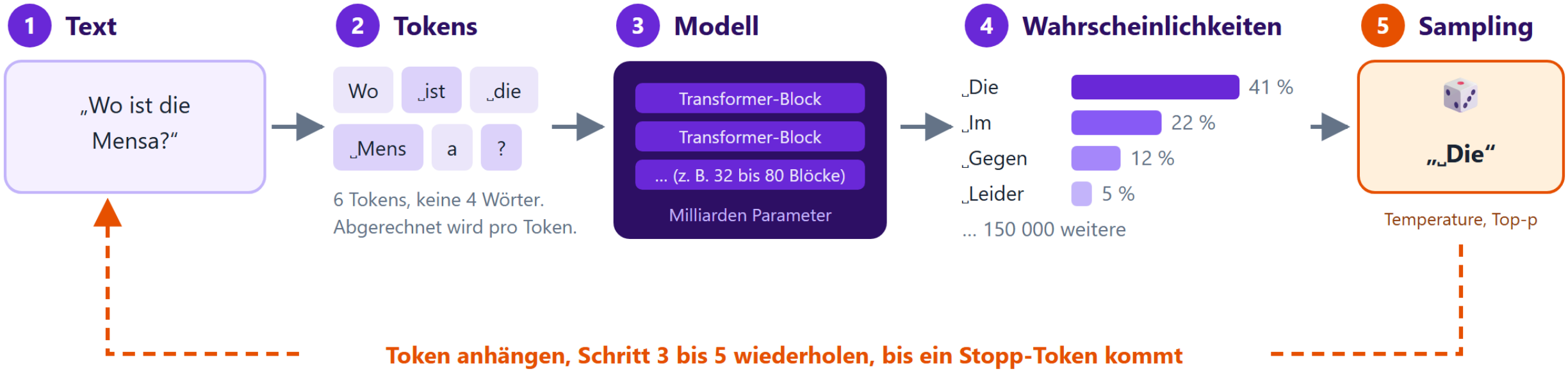
Konsequenz: Modelle **in eurer Sprache und mit euren Fragen** testen. Lokales, aktuelles Wissen müssen wir dem Modell zur Laufzeit mitgeben, das ist VL 4.

2 · Modell und Größe

Was zwischen Frage und Antwort passiert

Vom Text zur Antwort: ein Token pro Schritt

Was bei jedem Aufruf des Campus-Assistenten passiert



Prefill

Alle Eingabe-Tokens werden **parallel** verarbeitet.

Bestimmt die **Time to First Token** (TTFT).

Decode

Ein Ausgabe-Token nach dem anderen, **sequenziell**.

Bestimmt die **Time per Output Token** (TPOT).

Drei Folgen aus diesem Bild

🔁 Ein Token pro Schritt

Das Modell ist **autoregressiv**: Jedes neue Token baut auf allen vorigen auf, auch auf den eigenen. Es gibt kein „erst nachdenken, dann antworten“.

Prefill und Decode

Prefill liest die Eingabe parallel und bestimmt die **TTFT** aus VL 1. **Decode** erzeugt Token für Token und bestimmt die **TPOT**.

📏 Begrenzter Kontext

Für jedes bisherige Token speichert das Modell Zwischenergebnisse. Llama 2 schaffte 4 000 Tokens, Llama 3 schafft 128 000. Eingabe und Ausgabe müssen zusammen hineinpassen.

Den **Transformer** behandeln wir als Blackbox: ein Stapel gleichartiger Blöcke (Llama 3 8B: 32 Stück). Sein Kern, die **Attention**, darf bei jedem Token auf alle bisherigen Tokens schauen. Huyens Bild dafür: Ältere Architekturen mussten eine Prüfungsfrage mit der Zusammenfassung des Buchs beantworten, ein Transformer darf jede Seite nachschlagen.

Wie groß ist ein Modell, und was heißt das für den Betrieb?

Die Zahl im Namen sind die **Parameter**, also gelernte Gewichte. Faustregel für den Speicher:

Parameter × Bytes pro Parameter

qwen2.5:0.5b
0,5 Mrd. × 2 Byte = **1 GB**. Läuft auf jedem Laptop.

llama3.2:3b
3 Mrd. × 2 Byte = **6 GB**, mit 4-Bit-Quantisierung etwa 2 GB.

llama-3.1-8b
8 Mrd. × 2 Byte = **16 GB**. Eine gute Gaming-GPU.

llama-3.3-70b
70 Mrd. × 2 Byte = **140 GB**. Mehrere Rechenzentrums-GPUs.

Qwen: Alibaba · Llama: Meta. Dazu kommt Speicher für den Kontext. Größer ist innerhalb einer Familie meist besser, aber neue kleine Modelle schlagen oft alte große: Llama 3 mit 8 Mrd. übertrifft Llama 2 mit 70 Mrd.

Mixture of Experts: ein Router wählt die Experten

Ein **MoE-Modell** ersetzt in jedem Block die eine große Feed-Forward-Schicht durch **mehrere kleinere Experten** (FFN 1 bis 4).

- Ein kleiner **Router** schaut sich jedes Token an und schickt es zu einem oder wenigen Experten.
- Im Bild landet „More“ bei Experte 2, „Parameters“ bei Experte 1.
- Die anderen Experten **rechnen für dieses Token nicht mit**.

> Viel Wissen in den Parametern, aber pro Token nur ein Bruchteil der Rechenarbeit.

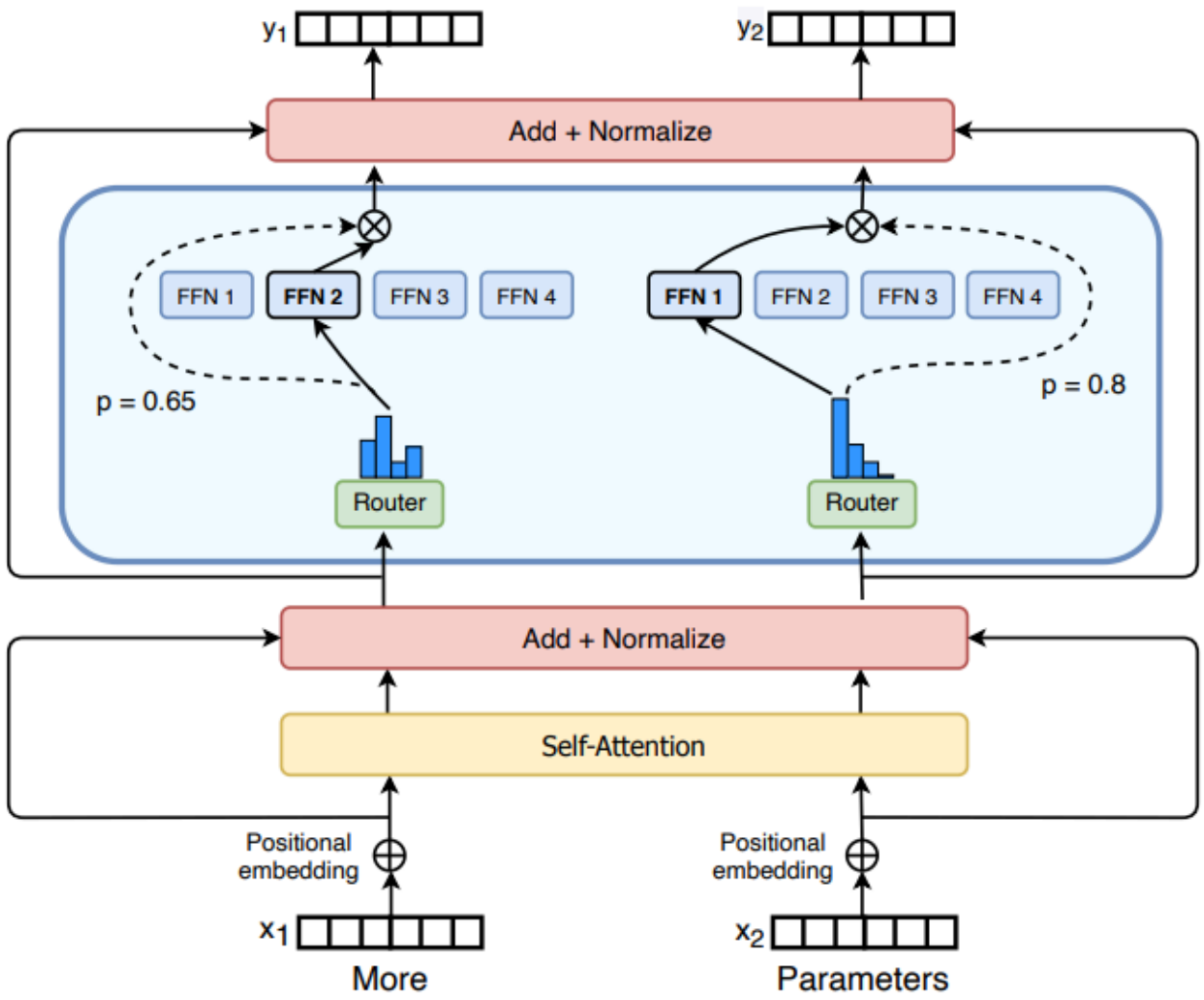


Abbildung: Fedus, Zoph, Shazeer: *Switch Transformers*, JMLR 2022, Abb. 2 (zugeschnitten), via Hugging Face Blog *Mixture of Experts Explained*, 2023.

Unser Cloud-Modell ist ein MoE-Modell

Unser Cloud-Modell, Name zerlegt:

qwen3 · 30b · a3b · instruct · 2507

- **qwen3**: Modellfamilie
- **30b**: rund 30 Mrd. Parameter insgesamt
- **a3b**: rund 3 Mrd. **aktiv** pro Token
- **instruct**: nachtrainiert, gleich mehr dazu
- **2507**: Stand Juli 2025

💾 Speicher

richtet sich nach **allen** 30 Mrd. Parametern

⏱️ Geschwindigkeit und Kosten

richten sich nach den **aktiven** 3 Mrd.

📖 Beispiel aus dem Buch

Mixtral 8x7B: 46,7 Mrd. gesamt, 12,9 Mrd. aktiv

3 · Post-Training

Vom Textvervollständiger zum Assistenten

Drei Trainingsstufen, drei verschiedene Modelle

Dieselbe Eingabe: „Wie melde ich mich zur Prüfung an“

📚 Pre-Training

Nächstes Token vorhersagen, Billionen Tokens aus dem Web

ca. 98 % der Rechenleistung

… an, wenn ich krank bin? Und was passiert, wenn ich die Frist verpasse? Ich habe im Forum gelesen, dass …

Ergebnis: Base-Modell

vervollständigt Text wie eine Webseite, beantwortet nichts

🎓 SFT

Supervised Finetuning auf Paaren aus Prompt und guter Antwort, von Menschen geschrieben

Du meldest dich im Portal deiner Hochschule an. Wähle dort die Prüfung aus und bestätige die Anmeldung.

Ergebnis: Instruct-Modell

folgt Anweisungen und führt ein Gespräch

👍 Präferenz-Finetuning

Menschen vergleichen zwei Antworten: welche ist besser?

RLHF, DPO, RLAIF

Das hängt von deiner Hochschule ab. Meist über das Prüfungsportal. Achte auf die Frist. Sag mir gern, wo du …

Ergebnis: Chat-Modell

hilfsbereit, vorsichtig, lehnt Schädliches ab

Was das für euch heißt

Am Modellnamen die Stufe erkennen:

- **Base** oder ohne Zusatz: setzt Text fort, beantwortet nichts. Für Anwendungen ungeeignet.
- **Instruct** oder **Chat**: SFT und meist Präferenz-Finetuning. **Die Variante für eure Anwendung.**

Präferenz-Finetuning gibt es in mehreren Varianten: *RLHF* mit eigenem Bewertungsmodell, *DPO* (einfacher, genutzt für Llama 3), *RLAIF* mit einem Modell als Bewerter.

Hilfsbereit heißt nicht wahrhaftig.

Beim Präferenz-Finetuning zählt, welche Antwort Menschen *lieber lesen*. Eine flüssige, selbstbewusste Antwort gewinnt oft, auch wenn sie falsch ist.

Das ist ein Teil der Erklärung für „private, staatlich anerkannte Hochschule“.

4 · Sampling

Wie aus Wahrscheinlichkeiten eine Antwort wird

Wie aus Zahlen eine Antwort wird: drei Schritte

Eingabe: **„Die HTWG Konstanz ist eine …“** Was kommt als nächstes Token?

① Logits

Das Modell gibt für **jedes** Token eine Rohzahl aus. Keine Prozente, nur „mehr“ oder „weniger passend“.

```
staatliche 3,2
private    1,5
Hochschule 0,9
…
```

→

② Softmax

macht daraus Wahrscheinlichkeiten, die zusammen 100 % ergeben.

```
staatliche 66 %
private    12 %
Hochschule  7 %
…
```

→

③ Ziehen 🎲

Wie ein Glücksrad, bei dem jedes Token so viel Fläche hat wie seine Wahrscheinlichkeit.

In 66 von 100 Fällen kommt **staatliche**,
in 12 von 100 Fällen **private**.

Greedy: Man könnte auch immer das wahrscheinlichste Token nehmen. Für Text wird das langweilig und repetitiv, deshalb wird normalerweise **gezogen**.

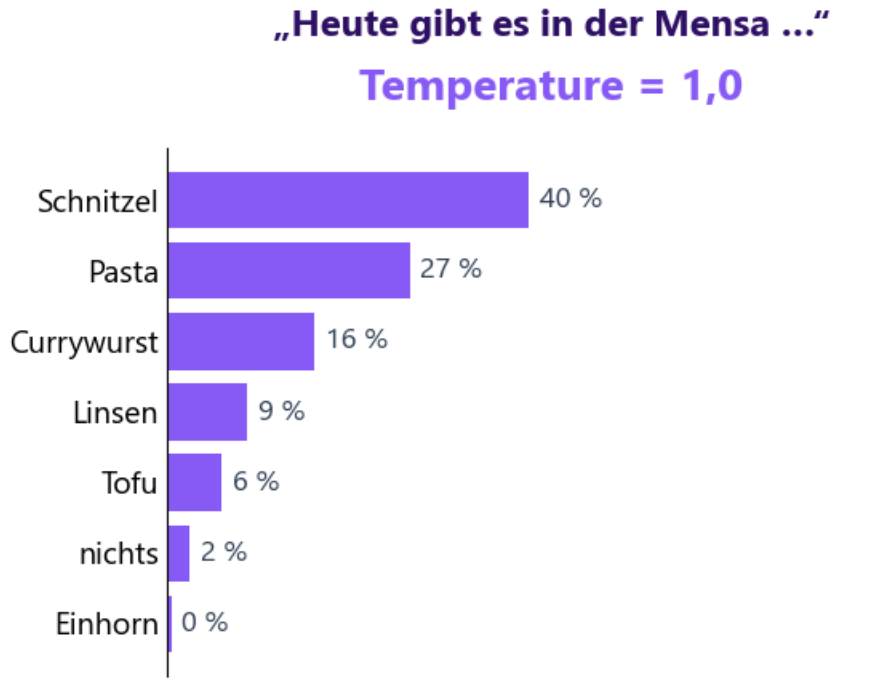
Die Frage vom Anfang: Das Modell „weiß“ meist das Richtige, zieht aber in etwa jedem achten Durchlauf „private“. Wie stark gezogen wird, stellt ihr ein.

Temperature: die Verteilung spitzer oder flacher machen

Die Logits werden vor der Softmax **durch die Temperature geteilt**.

- **0 bis 0,3**: fast immer das wahrscheinlichste Token. Für Extraktion, Klassifikation, alles, was ein Programm weiterverarbeitet.
- **um 0,7**: oft empfohlen für kreative Texte.
- **über 1**: seltene Tokens kommen nach oben. Kreativ, aber zunehmend wirr.

Huyens Beispiel: Logits 1 und 2. Bei Temperature 1 gewinnt B mit 73 %, bei 0,5 mit 88 %. Anbieter erlauben meist 0 bis 2.



1,0: die Verteilung, so wie das Modell sie berechnet

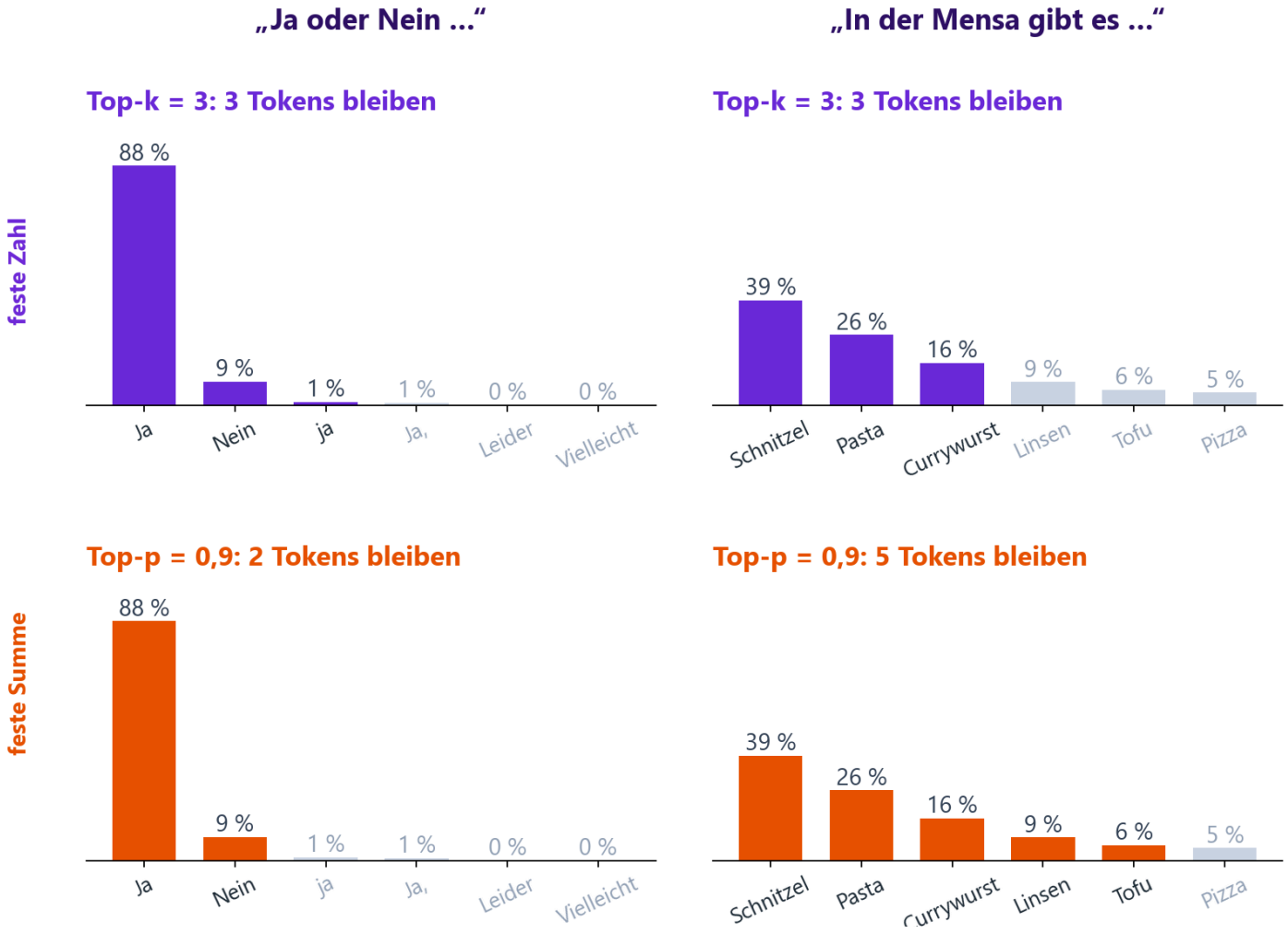
Top-k und Top-p: den Rand abschneiden

Vor dem Ziehen werden unwahrscheinliche Tokens gestrichen (grau).

Top-k: nur die **k** wahrscheinlichsten Tokens. Typisch 50 bis 500. Bei Ja/Nein bleibt dann Müll im Topf, beim Mensa-Gericht fliegen gute Kandidaten raus.

Top-p: so viele Tokens, bis ihre Summe **p** erreicht. Typisch 0,9 bis 0,95. **Passt sich an** die Verteilung an.

Dazu kommen **Stoppbedingungen**: ein Stopp-Token oder `max_tokens`. Zu knapp gesetzt, fehlt bei JSON die schließende Klammer.



Test-Time Compute: mehrmals ziehen, das Beste nehmen

Statt einer Antwort mehrere erzeugen und eine auswählen. Ausgewählt wird …

📈 nach Wahrscheinlichkeit

die Antwort mit der höchsten durchschnittlichen Token-Wahrscheinlichkeit

⚖️ mit einem Verifier

ein Bewertungsmodell wählt aus. Bei OpenAI brachte das so viel wie ein **30× größeres** Modell

🗳️ per Mehrheit

Self-Consistency: Gemini beantwortete jede MMLU-Frage 32-mal, die häufigste Antwort zählt

✅ per Regel

so lange erzeugen, bis die SQL-Abfrage oder das JSON gültig ist

Der Preis: Zwei Antworten kosten etwa doppelt so viel wie eine. Auch **Reasoning-Modelle** kaufen Qualität mit mehr Tokens pro Anfrage.

Strukturierte Ausgaben: von Pflaster bis Therapie

Eine Anwendung braucht oft **Daten statt Fließtext**: JSON mit festen Feldern, eine gültige Kategorie, SQL.

🩹 Prompting

„Antworte nur mit JSON." Funktioniert oft, garantiert nichts.

🔧 Nachbearbeitung

Typische Fehler per Code reparieren. LinkedIn: 90 % → 99,99 % gültiges YAML.

🔁 Neu erzeugen

Test-Time Compute: so lange, bis es passt.

🧩 Constrained Sampling

Tokens, die das Schema verletzen, werden vor dem Ziehen gestrichen. In der API: `response_format` mit JSON-Schema.

🏋️ Finetuning

Auf Beispiele im Zielformat nachtrainieren. Am zuverlässigsten, am aufwendigsten.

✅ Trotzdem validieren

Das Schema garantiert die **Form**, nicht den **Inhalt**.

Campus-Assistent: Anfragen strukturiert einordnen

```python
class Anfrage(BaseModel):
    anliegen: Literal["pruefung", "mensa",
                      "bibliothek", "sonstiges"]
    studiengang: str | None
    frist: str | None

antwort = client.chat.completions.create(
    model=os.environ["LLM_MODEL"],
    temperature=0, messages=nachricht,
    response_format=schema_von(Anfrage))
anfrage = Anfrage.model_validate_json(
    antwort.choices[0].message.content)
```

1. **`Anfrage`** beschreibt das Ergebnis. `Literal` erlaubt nur vier Kategorien, `None` heißt: darf fehlen.
2. **`temperature=0`**, denn eine Einordnung soll reproduzierbar sein.
3. **`response_format`** schickt das Schema mit, das Modell kann nur passende Ausgaben erzeugen.
4. **`model_validate_json`** prüft trotzdem. Bei Fehlern: erneut versuchen.

`schema_von` ist eine kleine Hilfsfunktion aus dem Starter, die das JSON-Schema der Klasse verpackt.

Jede Antwort ist eine Stichprobe

🔀 Inkonsistenz

Gleiche Eingabe, andere Ausgabe: kommt vom Zufall beim Sampling. Gegenmittel: niedrige Temperature, fester Seed, Caching. Keine Garantie, selbst die Hardware spielt mit.

Leicht andere Eingabe, ganz andere Ausgabe: schwerer. Gegenmittel: gute Prompts (VL 3), Evaluation (VL 8).

🌀 Halluzination

Zwei Hypothesen, die sich ergänzen:

Selbsttäuschung: Das Modell behandelt seine eigene Ausgabe wie eine Tatsache. Steht „private“ einmal da, baut es darauf auf *(Snowballing)*.

Fehlendes Wissen: Beim SFT lernt es, Antworten nachzuahmen, deren Wissen es selbst nicht hat.

Was wirklich hilft: dem Modell die Fakten **mitgeben**, statt sich auf sein Gedächtnis zu verlassen. Ab VL 4. Kurze Antworten senken das Risiko ebenfalls.

Jetzt ihr: 30 Minuten Sampling-Labor

Öffnet in Moodle **„VL 02 · Sampling-Labor“** und daneben **Google AI Studio** (aistudio.google.com).
Arbeitet zu zweit, notiert eure Antworten direkt im Labor.

Übung 1 ab 11:30: bis Montag, 19.10., 23:59

Übung 1 ist eure erste Praktikumsabgabe **P1**. Sie verbindet VL 1 und VL 2 und ist auf **etwa eine Woche** angelegt.

🐬 1 · Setup

Docker, eigenes Repo, erster Modellaufruf, TTFT messen

🔁 2 · Modell tauschen

nur über `.env`: `llama3.2:3b` gegen `qwen2.5:0.5b`

🎲 3 · Messen

Streuung bei drei Temperaturen, Tokens Deutsch gegen Englisch

🧩 4 · Einordnen

Mails strukturiert einordnen: Schema, Validierung, erneuter Versuch

📝 5 · README

Ergebnisse und Reflexion, das ist die Abgabe

➕ Zusatz

Ausgaben inhaltlich absichern. Entfällt bei 80 % Anwesenheit

Heute in der Übung: Teil 1 und 2 schaffen, damit Docker und die Modelle bei allen laufen. Den Rest macht ihr bis Montag. Alles funktioniert lokal, die Academic Cloud ist optional.

---|---|---|
A	Starter aktualisieren, Modell `llama3.2:3b` lokal, Academic Cloud optional	15 Min
B	Sampling messen: wie viele verschiedene Antworten bei Temperature 0, 0,7 und 1,2?	20 Min
C	Tokens zählen: Deutsch gegen Englisch	10 Min
D	Strukturierte Einordnung von Anfragen mit Pydantic, Validierung und erneutem Versuch	45 Min

Teil D ist der Kern der ersten Praktikumsabgabe P1. Was ihr heute baut, gebt ihr ab.

Quellen

- Chip Huyen: *AI Engineering. Building Applications with Foundation Models*. O'Reilly, 2025, Kapitel 2
- Roland Huß, Daniele Zonca: *Generative AI on Kubernetes*. O'Reilly, 2026, Introduction, *Understanding LLM Fundamentals*
- Ashish Vaswani et al.: *Attention Is All You Need*. NeurIPS, 2017
- Lai et al.: *ChatGPT Beyond English* (2023), Verteilung der Sprachen in Common Crawl (zitiert nach Huyen)
- Bottaro, Ramgopal: LinkedIn, *Defensive YAML Parsing* (2020, zitiert nach Huyen)
- William Fedus, Barret Zoph, Noam Shazeer: *Switch Transformers*. JMLR, 2022 (Abbildung MoE, via Hugging Face Blog *Mixture of Experts Explained*, 2023)
- Logos: Qwen und Meta, Wikimedia Commons

Das Modell würfelt. Eure Anwendung muss damit rechnen.

Nächste Woche: Prompt Engineering. Wie schreibe ich Prompts, die reproduzierbar und robust sind?